

Control Flow Programs

in sh (Bourne Shell) — UNIX

Based on GeeksForGeeks Control Flow C Programs

All programs use **plain sh** syntax only. Key rules:

- Use `echo` for output (standard UNIX)
- Use `read var` for user input
- Use `expr` for integer arithmetic
- Use `echo "..."` | `bc -l` for floating-point
- Use `if/elif/else/fi` for branching
- Use `while` loops — no `for(( ))` or `[[ ]]`

1. Check Whether a Number is Positive, Negative, or Zero

```
1 #!/bin/sh
2 echo "Enter a number:"
3 read n
4 if [ $n -gt 0 ]
5 then
6 echo "$n is Positive"
7 elif [ $n -lt 0 ]
8 then
9 echo "$n is Negative"
10 else
11 echo "$n is Zero"
12 fi
```

Output:

```
Enter a number:
-7
-7 is Negative
```

2. Check Whether Number is Even or Odd

```
1 #!/bin/sh
2 echo "Enter a number:"
3 read n
4 rem=$(expr $n % 2)
5 if [ $rem -eq 0 ]
6 then
7 echo "$n is Even"
8 else
```

```
9 echo "$n is Odd"
10 fi
```

Output:

```
Enter a number:
13
13 is Odd
```

3. Check Whether a Character is Vowel or Consonant

```
1 #!/bin/sh
2 echo "Enter a character:"
3 read ch
4 case "$ch" in
5 [aeiouAEIOU])
6 echo "$ch is a Vowel"
7 ;;
8 [a-zA-Z])
9 echo "$ch is a Consonant"
10 ;;
11 *)
12 echo "$ch is not an alphabet"
13 ;;
14 esac
```

Output:

```
Enter a character:
E
E is a Vowel
```

4. Find Largest Number Among Three Numbers

```
1 #!/bin/sh
2 echo "Enter first number:"
3 read a
4 echo "Enter second number:"
5 read b
6 echo "Enter third number:"
7 read c
8
9 if [ $a -ge $b ] && [ $a -ge $c ]
10 then
11 echo "Largest number is: $a"
12 elif [ $b -ge $a ] && [ $b -ge $c ]
13 then
14 echo "Largest number is: $b"
```

```
15 else
16 echo "Largest number is: $c"
17 fi
```

Output:

```
Enter first number:
12
Enter second number:
47
Enter third number:
33
Largest number is: 47
```

5. Calculate Sum of Natural Numbers

```
1 #!/bin/sh
2 echo "Enter a positive integer N:"
3 read n
4 sum=0
5 i=1
6 while [ $i -le $n ]
7 do
8 sum=$((expr $sum + $i))
9 i=$((expr $i + 1))
10 done
11 echo "Sum of first $n natural numbers = $sum"
```

Output:

```
Enter a positive integer N:
10
Sum of first 10 natural numbers = 55
```

6. Print Alphabets From A to Z Using Loop

```
1 #!/bin/sh
2 echo "Alphabets from A to Z:"
3 i=65
4 while [ $i -le 90 ]
5 do
6 # Convert ASCII value to character using printf and octal
7 printf "\\$(printf '%03o' $i) "
8 i=$((expr $i + 1))
9 done
10 echo ""
```

Output:

```
Alphabets from A to Z:  
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

7. Check Leap Year

```
1 #!/bin/sh  
2 echo "Enter a year:"  
3 read year  
4 rem4=$(expr $year % 4)  
5 rem100=$(expr $year % 100)  
6 rem400=$(expr $year % 400)  
7  
8 if [ $rem400 -eq 0 ]  
9 then  
10 echo "$year is a Leap Year"  
11 elif [ $rem100 -eq 0 ]  
12 then  
13 echo "$year is NOT a Leap Year"  
14 elif [ $rem4 -eq 0 ]  
15 then  
16 echo "$year is a Leap Year"  
17 else  
18 echo "$year is NOT a Leap Year"  
19 fi
```

Output:

```
Enter a year:  
2000  
2000 is a Leap Year
```

8. Find Factorial of a Number

```
1 #!/bin/sh  
2 echo "Enter a non-negative integer:"  
3 read n  
4 fact=1  
5 i=1  
6 while [ $i -le $n ]  
7 do  
8 fact=$(expr $fact \* $i)  
9 i=$(expr $i + 1)  
10 done  
11 echo "Factorial of $n = $fact"
```

Output:

```
Enter a non-negative integer:
6
Factorial of 6 = 720
```

9. Simple Calculator

```
1  #!/bin/sh
2  echo "Enter first number:"
3  read a
4  echo "Enter operator (+, -, *, /):"
5  read op
6  echo "Enter second number:"
7  read b
8
9  case "$op" in
10 +)
11 result=$(echo "scale=2; $a + $b" | bc -l)
12 ;;
13 -)
14 result=$(echo "scale=2; $a - $b" | bc -l)
15 ;;
16 \*)
17 result=$(echo "scale=2; $a * $b" | bc -l)
18 ;;
19 /)
20 if [ "$b" = "0" ]
21 then
22 echo "Error: Division by zero"
23 exit 1
24 fi
25 result=$(echo "scale=2; $a / $b" | bc -l)
26 ;;
27 *)
28 echo "Error: Invalid operator"
29 exit 1
30 ;;
31 esac
32 echo "$a $op $b = $result"
```

Output:

```
Enter first number:
10
Enter operator (+, -, *, /):
/
Enter second number:
4
10 / 4 = 2.50
```

10. Generate Multiplication Table

```
1  #!/bin/sh
2  echo "Enter a number:"
3  read n
4  echo "Multiplication table of $n:"
5  i=1
6  while [ $i -le 10 ]
7  do
8  result=$(expr $n \* $i)
9  echo "$n x $i = $result"
10 i=$(expr $i + 1)
11 done
```

Output:

```
Enter a number:
5
Multiplication table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

11. Print Fibonacci Series

```
1  #!/bin/sh
2  echo "Enter number of terms:"
3  read n
4  a=0
5  b=1
6  i=1
7  echo "Fibonacci Series:"
8  while [ $i -le $n ]
9  do
10 echo $a
11 temp=$(expr $a + $b)
12 a=$b
13 b=$temp
14 i=$(expr $i + 1)
15 done
```

Output:

```
Enter number of terms:
8
Fibonacci Series:
0
1
1
2
3
5
8
13
```

12. Find LCM of Two Numbers

```
1  #!/bin/sh
2  echo "Enter first number:"
3  read a
4  echo "Enter second number:"
5  read b
6
7  # LCM = (a * b) / GCD
8  # First compute GCD using Euclidean algorithm
9  x=$a
10 y=$b
11 while [ $y -ne 0 ]
12 do
13   temp=$y
14   y=$(expr $x % $y)
15   x=$temp
16 done
17 gcd=$x
18
19 lcm=$(expr $a \* $b / $gcd)
20 echo "LCM of $a and $b = $lcm"
```

Output:

```
Enter first number:
12
Enter second number:
18
LCM of 12 and 18 = 36
```

13. Check Armstrong Number

```
1  #!/bin/sh
2  echo "Enter a number:"
3  read n
```

```
4
5 # Count digits
6 temp=$n
7 digits=0
8 while [ $temp -gt 0 ]
9 do
10 digits=$((expr $digits + 1))
11 temp=$((expr $temp / 10))
12 done
13
14 # Sum of each digit raised to power = digits
15 temp=$n
16 sum=0
17 while [ $temp -gt 0 ]
18 do
19 digit=$((expr $temp % 10))
20 power=$((echo "$digit^$digits" | bc))
21 sum=$((expr $sum + $power))
22 temp=$((expr $temp / 10))
23 done
24
25 if [ $sum -eq $n ]
26 then
27 echo "$n is an Armstrong Number"
28 else
29 echo "$n is NOT an Armstrong Number"
30 fi
```

Output:

```
Enter a number:
153
153 is an Armstrong Number
```

14. Display Armstrong Numbers Between 1 to 1000

```
1 #!/bin/sh
2 echo "Armstrong numbers between 1 and 1000:"
3 n=1
4 while [ $n -le 1000 ]
5 do
6 temp=$n
7 digits=0
8 while [ $temp -gt 0 ]
9 do
10 digits=$((expr $digits + 1))
11 temp=$((expr $temp / 10))
12 done
13
14 temp=$n
```



```
15 sum=0
16 while [ $temp -gt 0 ]
17 do
18   digit=$(expr $temp % 10)
19   power=$(echo "$digit^$digits" | bc)
20   sum=$(expr $sum + $power)
21   temp=$(expr $temp / 10)
22 done
23
24 if [ $sum -eq $n ]
25 then
26   echo $n
27 fi
28 n=$(expr $n + 1)
29 done
```

Output:

```
Armstrong numbers between 1 and 1000:
1
2
3
4
5
6
7
8
9
153
370
371
407
```

15. Display Armstrong Numbers Between Two Intervals

```
1  #!/bin/sh
2  echo "Enter lower interval:"
3  read low
4  echo "Enter upper interval:"
5  read high
6
7  echo "Armstrong numbers between $low and $high:"
8  n=$low
9  while [ $n -le $high ]
10 do
11   temp=$n
12   digits=0
13   while [ $temp -gt 0 ]
14   do
15     digits=$(expr $digits + 1)
```

```
16 temp=$(expr $temp / 10)
17 done
18
19 temp=$n
20 sum=0
21 while [ $temp -gt 0 ]
22 do
23 digit=$(expr $temp % 10)
24 power=$(echo "$digit^$digits" | bc)
25 sum=$(expr $sum + $power)
26 temp=$(expr $temp / 10)
27 done
28
29 if [ $sum -eq $n ]
30 then
31 echo $n
32 fi
33 n=$(expr $n + 1)
34 done
```

Output:

```
Enter lower interval:
100
Enter upper interval:
500
Armstrong numbers between 100 and 500:
153
370
371
407
```

16. Reverse a Number

```
1 #!/bin/sh
2 echo "Enter a number:"
3 read n
4 rev=0
5 temp=$n
6 while [ $temp -gt 0 ]
7 do
8 digit=$(expr $temp % 10)
9 rev=$(expr $rev \* 10 + $digit)
10 temp=$(expr $temp / 10)
11 done
12 echo "Reversed number = $rev"
```

Output:

```
Enter a number:
```

```
12345
Reversed number = 54321
```

17. Check Whether a Number is a Palindrome or Not

```
1 #!/bin/sh
2 echo "Enter a number:"
3 read n
4 rev=0
5 temp=$n
6 while [ $temp -gt 0 ]
7 do
8 digit=$(expr $temp % 10)
9 rev=$(expr $rev \* 10 + $digit)
10 temp=$(expr $temp / 10)
11 done
12
13 if [ $rev -eq $n ]
14 then
15 echo "$n is a Palindrome"
16 else
17 echo "$n is NOT a Palindrome"
18 fi
```

Output:

```
Enter a number:
121
121 is a Palindrome
```

18. Check Whether a Number is Prime or Not

```
1 #!/bin/sh
2 echo "Enter a number:"
3 read n
4 if [ $n -le 1 ]
5 then
6 echo "$n is NOT Prime"
7 exit 0
8 fi
9 i=2
10 is_prime=1
11 while [ $i -le $(expr $n / 2) ]
12 do
13 rem=$(expr $n % $i)
14 if [ $rem -eq 0 ]
15 then
16 is_prime=0
17 break
```

```
18 fi
19 i=$(expr $i + 1)
20 done
21 if [ $is_prime -eq 1 ]
22 then
23 echo "$n is Prime"
24 else
25 echo "$n is NOT Prime"
26 fi
```

Output:

```
Enter a number:
29
29 is Prime
```

19. Display Prime Numbers Between Intervals

```
1 #!/bin/sh
2 echo "Enter lower interval:"
3 read low
4 echo "Enter upper interval:"
5 read high
6
7 echo "Prime numbers between $low and $high:"
8 num=$low
9 while [ $num -le $high ]
10 do
11 if [ $num -le 1 ]
12 then
13 num=$(expr $num + 1)
14 continue
15 fi
16 i=2
17 is_prime=1
18 while [ $i -le $(expr $num / 2) ]
19 do
20 rem=$(expr $num % $i)
21 if [ $rem -eq 0 ]
22 then
23 is_prime=0
24 break
25 fi
26 i=$(expr $i + 1)
27 done
28 if [ $is_prime -eq 1 ]
29 then
30 echo $num
31 fi
32 num=$(expr $num + 1)
```

```
33 done
```

Output:

```
Enter lower interval:
10
Enter upper interval:
40
Prime numbers between 10 and 40:
11
13
17
19
23
29
31
37
```

20. Check Whether the Input Number is a Neon Number

A **Neon Number** is a number where the sum of digits of the square of the number equals the number itself. Example: $9^2 = 81$, and $8 + 1 = 9$.

```
1 #!/bin/sh
2 echo "Enter a number:"
3 read n
4 sq=$(expr $n \* $n)
5 temp=$sq
6 digit_sum=0
7 while [ $temp -gt 0 ]
8 do
9 digit=$(expr $temp % 10)
10 digit_sum=$(expr $digit_sum + $digit)
11 temp=$(expr $temp / 10)
12 done
13 echo "Square of $n = $sq"
14 echo "Sum of digits of $sq = $digit_sum"
15 if [ $digit_sum -eq $n ]
16 then
17 echo "$n is a Neon Number"
18 else
19 echo "$n is NOT a Neon Number"
20 fi
```

Output:

```
Enter a number:
9
Square of 9 = 81
Sum of digits of 81 = 9
```

```
9 is a Neon Number
```

21. Find All Factors of a Natural Number

```
1 #!/bin/sh
2 echo "Enter a natural number:"
3 read n
4 echo "Factors of $n:"
5 i=1
6 while [ $i -le $n ]
7 do
8   rem=$(expr $n % $i)
9   if [ $rem -eq 0 ]
10  then
11    echo $i
12  fi
13  i=$(expr $i + 1)
14 done
```

Output:

```
Enter a natural number:
36
Factors of 36:
1
2
3
4
6
9
12
18
36
```

22. Sum of Fibonacci Numbers at Even Indexes up to N Terms

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

Even-indexed terms (0-based): index 0, 2, 4, 6 ...i.e. 0, 1, 3, 8, 21 ...

```
1 #!/bin/sh
2 echo "Enter number of terms N:"
3 read n
4 a=0
5 b=1
6 i=0
7 even_sum=0
8 while [ $i -lt $n ]
9 do
10  rem=$(expr $i % 2)
```

```

11 if [ $rem -eq 0 ]
12 then
13 even_sum=$(expr $even_sum + $a)
14 fi
15 temp=$(expr $a + $b)
16 a=$b
17 b=$temp
18 i=$(expr $i + 1)
19 done
20 echo "Sum of Fibonacci numbers at even indexes up to $n terms =
    $even_sum"

```

Output:

```

Enter number of terms N:
8
Sum of Fibonacci numbers at even indexes up to 8 terms = 12

```

Quick Reference: Control Flow in sh

Construct	sh Syntax
if-else	if [cond]; then ... elif ... else ... fi
case	case "\$var" in pattern) ... ;; esac
while loop	while [cond]; do ... done
break	break
continue	continue
Logical AND	[cond1] && [cond2]
Logical OR	[cond1] [cond2]
Modulo	expr \$a % \$b
Multiply	expr \$a * \$b
Power (bc)	echo "\$a^\$b" bc
Float arithmetic	echo "scale=2; expr" bc -l

Save as prog.sh, then run: `chmod +x prog.sh && ./prog.sh`